UDE@BOOK

Juan Camilo David Zapata
Juan Martin Lopez Gonzalez

Informatica II

Desafio 2

Informe Final

Semestre

2026-2

Universidad de Antioquia

Resumen

El presente informe describe el proceso de diseño e implementación de UDE@BOOK desarrollada en C++ mediante el uso de programación orientada a objetos, la solución se estructuró en seis clases principales: RedSocial, Usuario, Mensaje, Solicitudes, Album y Foto, cada una con responsabilidades específicas dentro del sistema.

1. Análisis del problema y consideraciones

1.1. El problema

Este desafio consiste en desarrollar un programa de consola que simule el funcionamiento de una red social mediante programación orientada a objetos en C++. El sistema debe permitir administrar usuarios, establecer relaciones de amistad, gestionar solicitudes, intercambiar mensajes y organizar fotografías en álbumes.

Además de implementar las funcionalidades de una red social, el desarrollo contempla aspectos relacionados con el manejo de memoria dinámica, la organización de la información mediante clases, el almacenamiento de datos en archivos y la evaluación de los recursos utilizados por el programa.

1.2. Consideraciones tomadas antes de programar

Antes de implementar las funcionalidades, se tuvieron en cuenta varias decisiones de diseño que permitieran organizar el código y facilitar su desarrollo.

Programación orientada a objetos. El programa se hizo mediante clases, agrupando sus atributos y operaciones.

Separación de responsabilidades La clase RedSocial se encarga de administrar el conjunto de usuarios y las relaciones de amistad, por otro lado, Usuario almacena la información individual y administra sus álbumes, mensajes enviados, mensajes recibidos y solicitudes, las clases restantes son los elementos que forman parte de estas estructuras.

Control de memoria Se implementaron constructores, constructores de copia, operadores de asignación y destructores en las clases, aunque algunas clases no administren memoria dinámica directamente o requieren operaciones de copia explícitas. El propósito es permitir la creación, copia y destrucción de objetos que contienen la información almacenada.

Representacion de amistades Elegimos una matriz de adyacencia de tipo bool para representar las relaciones entre los usuarios, cada posición de la matriz indica si existe una relación de amistad entre dos usuarios.

Evaluacion de recursos Se añadieron mecanismos para contabilizar determinadas iteraciones de los algoritmos y calcular el consumo de memoria.

1.3. Arquitectura

La arquitectura se organizó mediante seis clases: RedSocial, Usuario, Album, Foto, Mensaje y Solicitudes.

la organizacion de las clases de muestra de la siguiente manera

RedSocial: administra los usuarios y las relaciones de amistad.

Usuario: contiene los datos personales y administra las colecciones de álbumes, mensajes y solicitudes.

Album: organiza las fotografías y controla la privacidad del álbum.

Foto: almacena los datos de cada fotografía y permite modificar sus likes.

Mensaje: representa los mensajes enviados entre usuarios y registra si han sido leídos.

Solicitudes: representa las solicitudes de amistad recibidas por un usuario.

2. Esquema de tareas definidas para el desarrollo

Para desarrollar el sistema, se dividió el problema en diferentes clases, cada una para cumplir una responsabilidad concreta y proporcionar operaciones que las demas clases utilizaran.

2.1. RedSocial

Es la clase encargada de coordinar el funcionamiento general. Administra el conjunto de usuarios, mantiene las relaciones de amistad y proporciona las operaciones que permiten interactuar. Entre sus principales responsabilidades se encuentran:

- Registrar usuarios y buscar usuarios existentes.
- Validar el inicio de sesión.
- Enviar, aceptar y rechazar solicitudes de amistad.
- Consultar amigos en común y generar sugerencias de amistad.
- Permitir la visita a perfiles de amigos y la consulta de sus álbumes visibles.
- Enviar mensajes y consultar mensajes nuevos o comprendidos en un rango de fechas.

2.2. Usuario

representa a cada integrante de la RedSocial. Almacena su nombre de usuario, contraseña, fecha de nacimiento, correo electrónico, ruta de la fotografía de perfil, ciudad, país y fecha de inscripción.
Además de sus datos personales, también administra:

- Los álbumes del usuario.
- Los mensajes enviados.
- Los mensajes recibidos.
- Las solicitudes de amistad recibidas.

2.3. Album

Representa una colección de fotografías del usuario. Sus atributos son el nombre del álbum, la ruta de almacenamiento, el nivel de privacidad y el arreglo dinámico de fotografías.
Dicha clase nos permite:

- Agregar fotografías.
- Modificar el nivel de privacidad.
- Mostrar el contenido del álbum.
- Ampliar la capacidad del arreglo cuando es necesario.

2.4. Foto

Representa cada fotografía almacenada en un álbum, contiene la fecha de la fotografía, su dimensión, la ruta del archivo y la cantidad de likes.

Sus operaciones permiten consultar y modificar los atributos correspondientes, incrementar o disminuir los likes y comparar fotografías mediante su ruta y fecha.

El método quitar-like() verifica que la cantidad de likes sea mayor a cero antes de disminuirla, así evitamos que el contador tome valores negativos al utilizar esta operación.

2.5. Mensaje

Representa la comunicación entre dos usuarios. Almacena el emisor, el receptor, el texto, la fecha de envío y un indicador que determina si el mensaje ha sido leído.

Sus principales operaciones permiten consultar la información del mensaje, modificar su estado de lectura y verificar determinadas condiciones.

2.6. Solicitudes

representa una solicitud de amistad, contiene el nombre del usuario emisor y un campo denominado fecha-envio, utilizado en la implementación para almacenar la información asociada a la solicitud.

Permite crear, copiar y asignar solicitudes, consultar sus atributos y comparar dos objetos, ademas la administración de las solicitudes se complementa con los métodos de Usuario que permiten agregarlas, buscarlas por el nombre del emisor y eliminarlas

2.7. Integracion de las clases

La integración se diseñó de manera que cada clase proporcione las operaciones necesarias para las demás sin necesidad de asumir todas las responsabilidades del sistema.

Por ejemplo, cuando se envía un mensaje, RedSocial valida los índices de los usuarios, busca al receptor y verifica que exista una amistad. Luego, crea un objeto Mensaje y solicita al emisor y al receptor que lo almacenen en sus respectivas colecciones.

3. Algoritmos implementados

En esta sección se describen los procedimientos identificados en los archivos, ademas, Se explica brevemente su funcionamiento y la forma en que contribuyen a las operaciones.

3.1. Registro y busqueda de usuarios

El registro de usuarios se realiza mediante RedSocial::agregar-usuario(). Antes de almacenar un nuevo usuario, el método consulta si ya existe otro con el mismo nombre de usuario.

Para realizar esta comprobación se utiliza Buscar-usuario(), que recorre los usuarios almacenados hasta encontrar una coincidencia. Si no encuentra el nombre solicitado, devuelve -1. Si el usuario ya existe el registro se rechaza, en caso contrario, se verifica si el arreglo dispone de espacio suficiente.

3.2. Ampliacion de arreglos dinamicos

Las clases RedSocial, Usuario y Album utilizan arreglos dinámicos para almacenar colecciones de tamaño variable.

El procedimiento consiste en comprobar si la cantidad de elementos almacenados ha alcanzado la capacidad disponible. Cuando esto ocurre, se crea un arreglo más grande, se copian los elementos existentes, se libera la memoria del arreglo anterior y se actualiza el puntero.

3.3. Consulta de amistades

Las relaciones de amistad se almacenan en una matriz de adyacencia de valores booleanos.

Para establecer una amistad se utiliza el método establecer-amistad() marca como verdaderas las dos posiciones correspondientes a los usuarios involucrados, para consultar una relación el metodo son-amigos() verifica los índices y devuelve el valor almacenado en la matriz.

La matriz se amplía cuando aumenta la capacidad de usuarios, durante esta operación se conservan las relaciones existentes y las nuevas posiciones se inicializan en false.

3.4. Calculo de amigos en comun

El método contar-amigos-comun-interno() recorre todos los usuarios y comprueba cuáles aparecen como amigos de los dos usuarios analizados.

Para cada índice (k), se verifica simultáneamente:

- Que el primer usuario sea amigo del usuario (k).
- Que el segundo usuario también sea amigo del usuario (k).

Cuando ambas condiciones se cumplen el contador de amigos en común aumenta en una unidad. Al finalizar el recorrido se devuelve el total obtenido.

Este procedimiento se utiliza en la generación de sugerencias de amistad y en la asignación de pesos para seleccionar a los mejores amigos.

3.5. Generacion sugerencias de amistad

El método mostrar-sugerencias-amistad() genera sugerencias para un usuario a partir de los amigos en común.

Primero recorre la lista de usuarios y selecciona aquellos que no son el usuario actual ni tienen una relación de amistad directa con él, para cada candidato calcula el número de amigos en común y almacena el índice del usuario y el resultado en arreglos dinámicos auxiliares.

Posteriormente aplica un ordenamiento por selección, en cada posición busca al candidato con la mayor cantidad de amigos en común y lo intercambia con el elemento ubicado en esa posición.

3.6. gestion de solicitudes de amistad

El envío de solicitudes se realiza mediante enviar-solicitud-amistad(). El método verifica que los índices de los usuarios sean válidos, que no se trate del mismo usuario y que los involucrados no sean ya amigos.

Si las comprobaciones se cumplen, se crea una solicitud con el nombre del emisor y se agrega a la colección de solicitudes recibidas del destinatario.

3.7. Envio y lectura de mensajes

El método enviar-mensaje() de RedSocial valida el índice del emisor, busca al receptor y comprueba que ambos usuarios sean distintos y tengan una relación de amistad.

Si se cumplen las condiciones se construye un objeto Mensaje con el emisor, el receptor, el texto, la fecha y el estado inicial de lectura establecido en false.

El mensaje se almacena en la colección de mensajes enviados del emisor y en la colección de mensajes recibidos del receptor, de esta forma cada usuario dispone de una copia del mensaje en su respectiva colección.

3.8. consulta de perfiles

La consulta de perfiles se realiza mediante visitar-perfil-amigo(). El método busca al usuario solicitado y comprueba que exista una relación de amistad con el visitante. Si el usuario no existe o no es amigo, se cancela la consulta.

Cuando se cumplen las condiciones, se recorren los álbumes del usuario visitado y se verifica su privacidad mediante album-visible().

4. Problemas de desarrollo que se afrontaron

Organizacion de las relaciones entre usuarios

Nuestro diseño consistió en representar las amistades y mantener coherentes las operaciones que dependen de ellas.

Para resolverlo se utilizó una matriz de adyacencia que permite consultar las relaciones entre usuarios, la operación que establece una amistad modifica las dos posiciones correspondientes de manera que la relación se mantiene simétrica.

Esta decisión también nos facilita el cálculo de amigos en común la generación de sugerencias y la validación de determinadas operaciones como el envío de mensajes.

Coodinacion entre las clases

La integración de las diferentes funcionalidades requiere que las clases colaboren sin concentrar toda la lógica en un único componente.

Por ejemplo, el envío de un mensaje requiere que RedSocial valide la relación entre los usuarios, que Mensaje represente la información de la comunicación y que Usuario almacene el mensaje en la colección correspondiente.

Algoritmos de seleccion

La generación de sugerencias y la selección de mejores amigos requieren procedimientos diferentes.

En el primer caso se ordenan los candidatos según el número de amigos en común. En el segundo se asignan pesos según ese mismo criterio

La implementación de ambos permite distinguir entre una recomendación basada en el orden de los resultados y una selección que incorpora aleatoriedad.

Validacion de las operaciones

Las operaciones de la red social deben comprobar determinadas condiciones antes de modificar los datos almacenados, por ejemplo no se permite enviar un mensaje al propio usuario, enviar mensajes a usuarios inexistentes o establecer una amistad mediante índices fuera del rango de usuarios almacenados.

Para estas situaciones se incorporaron validaciones en diferentes métodos de RedSocial así como comprobaciones de índices en los métodos de acceso a las colecciones.

5. Evolución de la solución y consideraciones de implementación

5.1. Planteamiento inicial

En la etapa inicial se elaboró un diagrama de clases que identificaba las entidades principales de la red social y sus relaciones, con este diseño permitió establecer una primera organización de los usuarios, los mensajes, las solicitudes, los álbumes y las fotografías.

Sin embargo, el diagrama inicial constituía una representación general del problema, durante la implementación se hizo necesario detallar los atributos y las operaciones

5.2. Integracion de funcionalidades

La solución se estructuró alrededor de RedSocial que coordina las operaciones principales y utiliza los métodos de las demás clases para ejecutar las acciones solicitadas.

Esta organización nos permite desarrollar funcionalidades compuestas mediante la colaboración entre objetos, el envío de mensajes, por ejemplo, utiliza la información de los usuarios, crea un objeto Mensaje y lo incorpora a las colecciones del emisor y del receptor.

5.3. Consideraciones finales

La solución utiliza arreglos dinámicos administrados manualmente, por lo que el correcto funcionamiento del programa depende de la gestión de sus capacidades, copias y liberación de memoria.

También depende de que los datos cumplan determinadas condiciones, por ejemplo, las fechas utilizadas en las comparaciones deben mantener un formato uniforme y las operaciones de envío deben aplicar las validaciones que correspondan.

Conclusión

El desarrollo de UDE@BOOK permitió estructurar mediante programación orientada a objetos, distribuyendo las responsabilidades entre seis clases principales, esta organización facilita representar los usuarios y sus relaciones, administrar contenidos y coordinar las operaciones de comunicación.

La implementación incorpora mecanismos de memoria dinámica, una matriz de adyacencia para las amistades, algoritmos para generar sugerencias y seleccionar mejores amigos, gestión de solicitudes, envío de mensajes, control de privacidad de álbumes y operaciones de almacenamiento en archivos.